

Dependency Risk Radar: A Multicriteria Supply Chain Security Tool for Android and Java Projects

Faik Mohammed Amine^a, Fath Othmane^a, Ikherazen Maryam^a, Isedrhas Aya^a, Pr Lachgar Mohammed^a

^a*Cadi Ayyad University, National School of Applied Sciences, Marrakech, Morocco*

Abstract

Software supply chain attacks have grown dramatically, making dependency auditing a critical yet under-tooled phase of secure software development. We present *Dependency Risk Radar* (DRR), an open-source platform that automatically audits Android and Java project dependencies across four risk dimensions: CVE exposure, version obsolescence, licence compliance, and embedded tracker detection. DRR ingests Gradle build files and compiled APKs, enriches each component through five external data sources in parallel, propagates risk transitively through a directed acyclic graph (DAG), and generates standards-compliant Software Bills of Materials in CycloneDX 1.5 and SPDX 2.3 formats. An integrated AI module produces a prioritised, migration-aware update plan. The tool exposes a REST/WebSocket API, an interactive React dashboard with D3.js dependency graph visualisation, and a CLI for local use.

Keywords: software supply chain security, SBOM, dependency analysis, Android security, static analysis, vulnerability scoring

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0.0
C2	Permanent GitHub link to code/repository used for this code version	https://github.com/Fath-oth/Dependency-Risk-Radar
C3	Legal Code License	MIT License
C4	Code versioning system used	Git
C5	Software code languages, tools, and services used	Python 3.11, FastAPI, NetworkX, Androguard, React 18, D3.js, Vite, TailwindCSS
C6	Compilation requirements, operating environments & dependencies	Python $\geq$ 3.11, Node.js $\geq$ 18, Docker (recommended); see <code>requirements.txt</code> and <code>package.json</code>
C7	Link to developer documentation/manual	https://github.com/Fath-oth/Dependency-Risk-Radar/blob/main/README.md
C8	Support email for questions	m.faik2230@uca.ac.ma

Table 1: Code metadata (mandatory)

1. Motivation and Significance

Software supply chain security has become a critical concern in modern development. High-profile incidents such as SolarWinds (2020), Log4Shell (2021), and XZ Utils (2024) demonstrated that compromising a single widely-used library can cascade to thousands of downstream applications. In the Android ecosystem specifically, studies show that over 80% of codebases contain at least one dependency with a known critical vulnerability [1]. Three risk vectors coexist in Android projects: explicitly declared Maven/Gradle dependencies, *transitive dependencies* (dependencies of dependencies, often invisible to developers), and third-party SDKs bundled directly in the APK. A typical Android project may declare 5–10 direct dependencies yet embed 50–200 transitive ones in its final binary. Existing tools address only parts of this problem. OWASP Dependency-Check lacks native Android support; Snyk and Dependabot are paid SaaS products without tracker detection; apkanalyzer produces a raw inventory with no risk scoring. **Dependency Risk Radar (DRR)** fills these gaps by combining automated SBOM generation, multi-criteria risk scoring, transitive risk

propagation via a dependency graph, and an AI-powered update planner — all in under 60 seconds.

2. Software Description

2.1. Overview of the Architecture

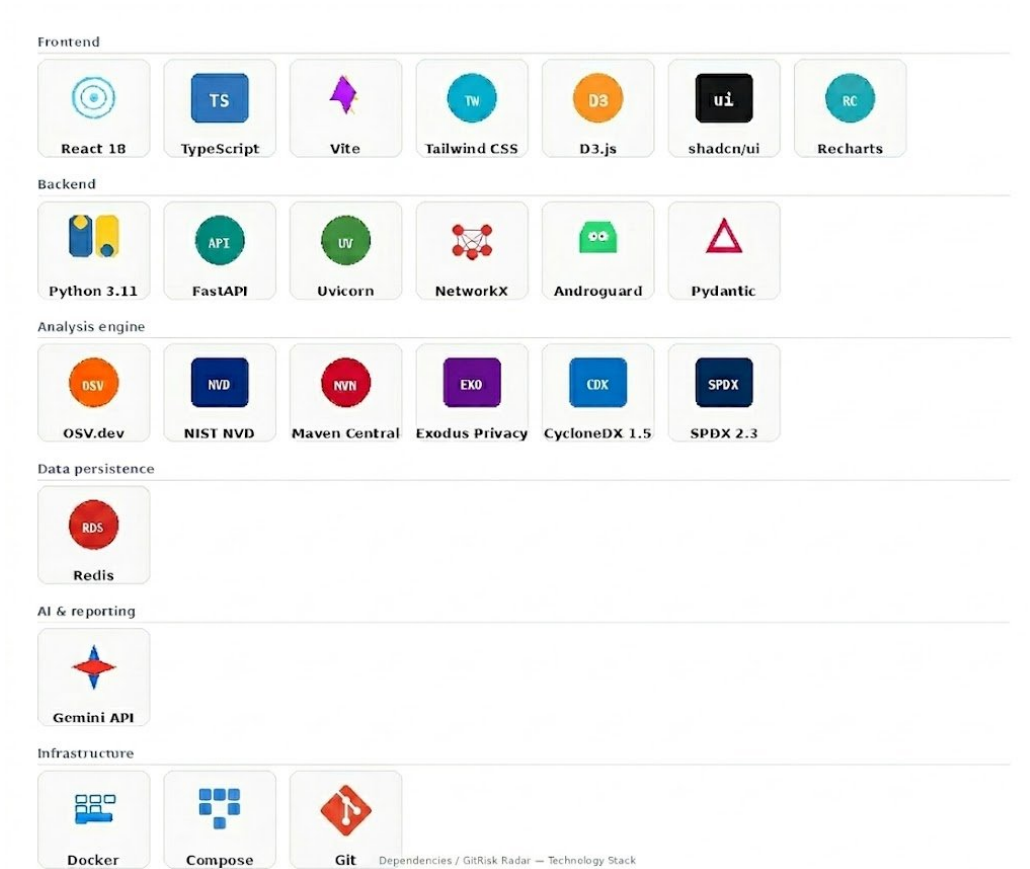


Figure 1: DRR technology stack: frontend (React 18, TypeScript, Vite, Tailwind CSS, D3.js, shadcn/ui, Recharts), backend (Python 3.11, FastAPI, Uvicorn, NetworkX, Androguard, Pydantic), analysis engine (OSV.dev, NIST NVD, Maven Central, Exodus Privacy, CycloneDX 1.5, SPDX 2.3), data persistence (Redis, SQLAlchemy), AI layer (Gemini API), and Docker infrastructure.

DRR is structured into four decoupled layers communicating through well-defined interfaces, as illustrated in Figure 2.

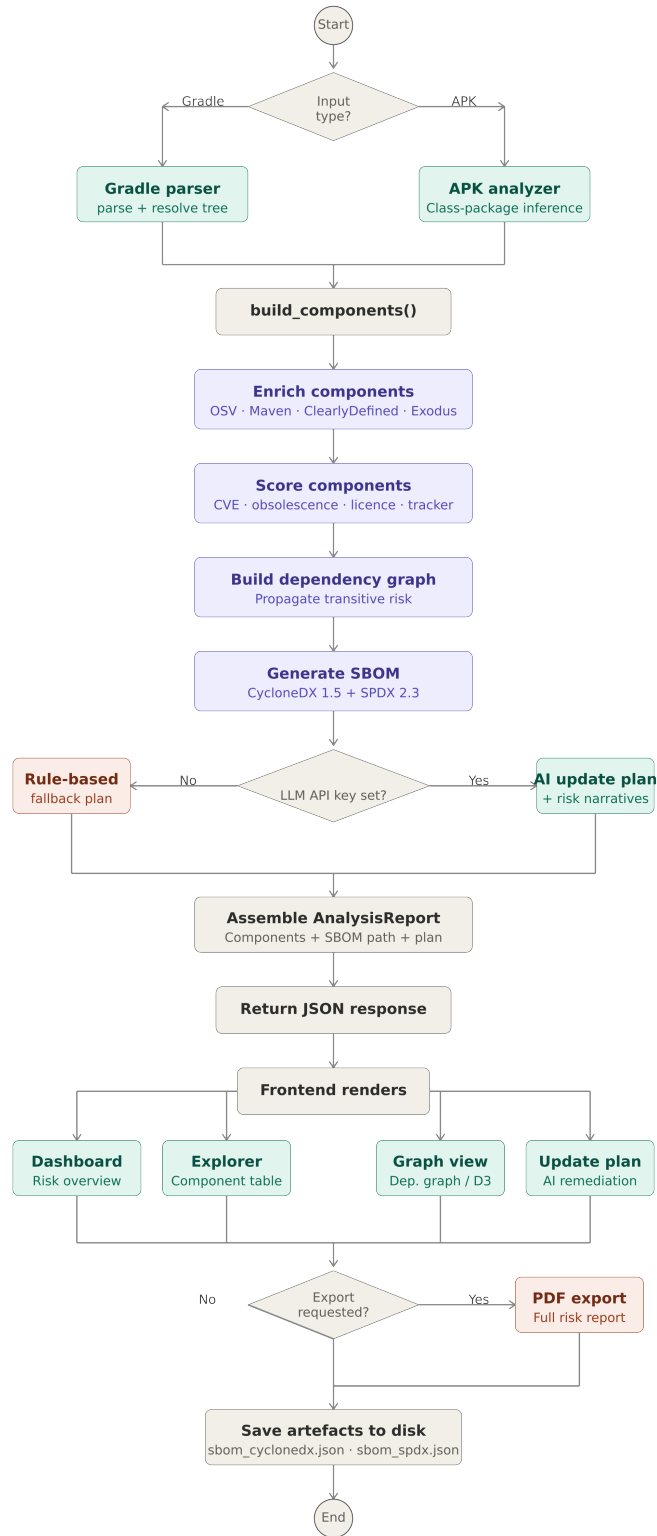


Figure 2: Global processing pipeline of Dependency Risk Radar.

Table 2: System layers and their responsibilities.

Layer	Responsibility	Key Technologies
1 — Ingestion	Gradle parsing, APK analysis, external API calls	Python, androguard
2 — Analysis	Multi-criteria scoring, dependency graph	Python, networkx
3 — AI	Update planning, narrative reports	Gemini Flash API
4 — Presentation	Dashboard, REST API, exports	FastAPI, React, D3.js

The end-to-end pipeline flows as follows:

- 0.1 The user submits a Gradle project directory or APK file via web UI.
- 0.2 **Layer 1** parses the sources and emits a list of raw components with metadata.
- 0.3 The dependency resolver invokes Gradle to obtain the full dependency tree (direct + transitive).
- 0.4 An enrichment step queries OSV.dev, NVD, Maven Central, and Exodus Privacy for each component.
- 0.5 **Layer 2** computes a numerical risk score for each component across four dimensions.
- 0.6 A dependency graph propagates transitive scores and identifies critical risk paths.
- 0.7 **Layer 3** generates a prioritized update plan and natural-language explanations.
- 0.8 **Layer 4** serves an interactive dashboard and produces exportable reports.

2.2. Backend Infrastructure

Gradle Parser.. The parser handles both Groovy DSL (`.gradle`) and Kotlin DSL (`.gradle.kts`) files, including version catalog references (`gradle/libs.versions.toml`). It extracts explicitly declared dependencies using a regular expression that captures the configuration scope (`implementation`, `api`, `kapt`, etc.), group ID, artifact ID, and version string.

To retrieve *transitive* dependencies, the parser invokes:

```
./gradlew :app:dependencies --configuration releaseRuntimeClasspath
```

It then parses the indented tree output (prefixes `+-`, `|`, `\--`) to reconstruct parent-child relationships and build the directed dependency graph.

APK Analyzer.. When source code is unavailable, DRR analyses the compiled APK directly using the Python library **androguard**. It extracts: declared permissions, min/target SDK, Dalvik bytecode class names (used for tracker detection), native **.so** libraries, and JAR signatures. This path is also used to *validate* the Gradle-derived SBOM against what is actually present in the binary.

SBOM Generation.. Every analysis produces a **Software Bill of Materials** in CycloneDX 1.5 (JSON/XML) and SPDX 2.3 (JSON/tag-value) formats. Each component entry includes: name, version, Package URL (PURL), SPDX licence identifier, SHA-256 hash, and dependency relationships. The SBOM is the central artefact consumed by all downstream layers.

Risk Scoring Engine.. Each component receives a global risk score computed as a weighted average of four sub-scores, all normalised to [0, 100]:

$$Risk = 0.45 \times S_{CVE} + 0.25 \times S_{obs} + 0.20 \times S_{lic} + 0.10 \times S_{track} \quad (1)$$

- **CVE score** (45%): derived from the maximum CVSS v3.1 score across all known vulnerabilities, queried from OSV.dev. A penalty of up to +15 points is applied for unfixed vulnerabilities, and up to +10 points for a high CVE count.
- **Obsolescence score** (25%): combines semantic version lag (major/minor/patch) versus the latest release on Maven Central, and the age of the project’s last published release (up to +50 points if inactive for more than two years).
- **Licence score** (20%): maps the SPDX licence identifier to a predefined risk table ranging from 0 (MIT, Apache-2.0) to 90 (AGPL-3.0) or 65 (unknown), reflecting legal contamination risk.
- **Tracker score** (10%): cross-references Dalvik class names against the Exodus Privacy database of 300+ known Android trackers, weighted by tracker category (analytics vs. fingerprinting vs. surveillance).

Score thresholds determine the recommended action:

Dependency Graph and Transitive Risk.. DRR models all components as a Directed Acyclic Graph (DAG) using **networkx**. Three graph algorithms are applied:

Table 3: Risk levels and recommended actions.

Score	Level	Action
0–19	Low	Standard monitoring
20–49	Moderate	Schedule update in next sprint
50–74	High	Update within 2 weeks
75–89	Critical	Immediate update or mitigation
90–100	Blocking	Block deployment, fix before release

- 0.1 **High-impact detection:** components with high in-degree (many dependents) are flagged as priority update candidates, since a single update propagates fixes to multiple dependants.
- 0.2 **Critical risk path finding:** for each vulnerable transitive library, `nx.shortest_path` traces back to the direct dependency that pulls it in, making the required action immediately actionable.
- 0.3 **Aggregated transitive score:** the transitive risk of a direct dependency is the maximum descendant score decayed exponentially with graph depth ($score \times 0.8^{d-1}$), so that a vulnerability one hop away is weighted more heavily than one five hops away.

2.3. Frontend Composition

The React 18 frontend communicates with the FastAPI backend via REST and WebSocket (for live progress streaming during analysis). It provides six main views:

- **Overview:** global risk gauge, severity distribution, top-5 critical components.
- **Component explorer:** filterable, sortable table with per-dimension scores.
- **Dependency graph:** D3.js force-directed layout; nodes coloured by risk level, sized by dependent count; click reveals the critical risk path.
- **Update plan:** AI-generated prioritised list with effort estimates and migration notes.
- **SBOM viewer:** interactive JSON/XML explorer of the generated SBOM.
- **History:** score evolution across successive analyses.

2.4. Database Management

In development mode, DRR uses an embedded SQLite database for zero-configuration deployment. Production deployments target PostgreSQL with SQLAlchemy (FastAPI-native ORM). A Redis cache layer stores LLM responses and external API results for 24 hours, avoiding redundant network calls and reducing analysis latency for repeated queries. The central data model is the **Component** object, which carries all enriched metadata (PURL, scores, CVE list, trackers, graph depth) and is persisted at the end of each analysis run as part of an **AnalysisReport** record.

2.5. Software Functionalities

Input modes.. DRR accepts two input types: (1) a Gradle project directory, in which case the Gradle parser extracts declared and transitive dependencies; and (2) a compiled APK file, which is analysed with **androguard** for class-level package inference. Both paths converge at the **build_components()** function that produces the unified component list.

SBOM export.. The generated SBOM is available for download in CycloneDX JSON, CycloneDX XML, and SPDX JSON/tag-value formats via the REST endpoint **GET /api/v1/reports/{id}/sbom**. It is also saved to disk as **sbom_cyclonedx.json** and **sbom_spdx.json** at the end of every run.

AI Update Planner.. The AI module sends the top-30 highest-risk components (score > 20) to the Gemini API with a structured system prompt. The model returns a JSON array of update recommendations, each specifying: current and recommended version, priority level (**CRITIQUE|HAUTE|MOYENNE|BASSE**), breaking-change risk, estimated migration effort, and optional migration notes. An executive summary paragraph is also generated for management reporting. If the LLM API key is absent or the call fails after three retries, a deterministic rule-based fallback plan is produced from the scoring data alone.

PDF report export.. On request, the backend renders the full analysis — SBOM summary, per-component scores, AI update plan, and risk narratives — into a downloadable PDF report suitable for security audits and compliance reviews.

3. Illustrative Examples

We demonstrate DRR on three publicly available, intentionally vulnerable Android benchmarks widely used in mobile security research: DIVA Android [2], OWASP UnCrackable Level 2 [3], and DamnVulnerableBank [4]. All experiments were conducted on a local workstation running DRR v1.0 via Docker Compose, with no modifications to the target projects.

3.1. Live Analysis Workflow

Figure 3 shows the landing screen during an active analysis session. The user specifies a server-side Gradle project path and clicks *Analyse Project*. DRR immediately streams real-time progress via Server-Sent Events (SSE): dependency extraction, parallel CVE enrichment, scoring, and SBOM generation are reported step-by-step, with a percentage counter updating continuously.

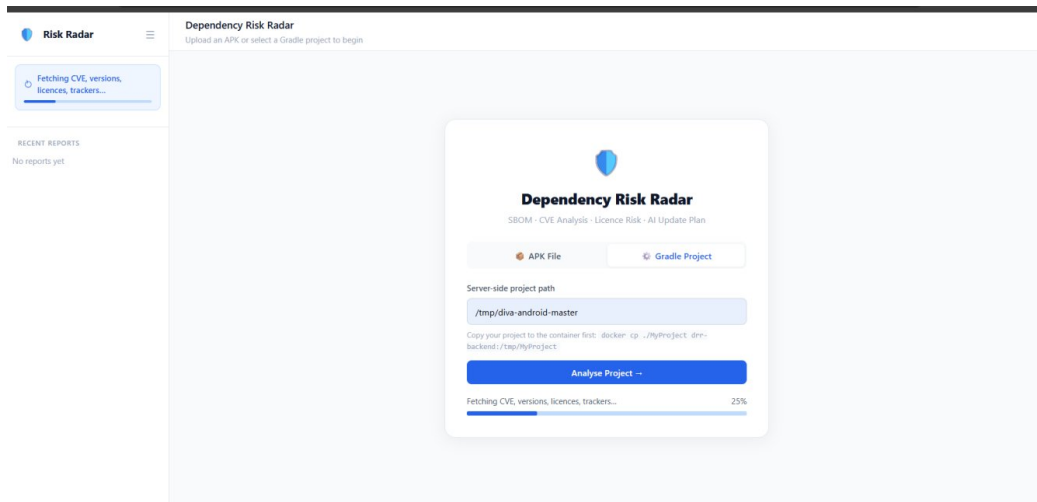


Figure 3: DRR landing screen with real-time SSE progress bar during a live CVE fetch on the `diva-android-master` Gradle project (25% complete).

3.2. DIVA Android

DIVA (*Damn Insecure and Vulnerable App*) is a classic Android learning target featuring intentional OWASP-category vulnerabilities. Its Gradle build declares three direct dependencies.

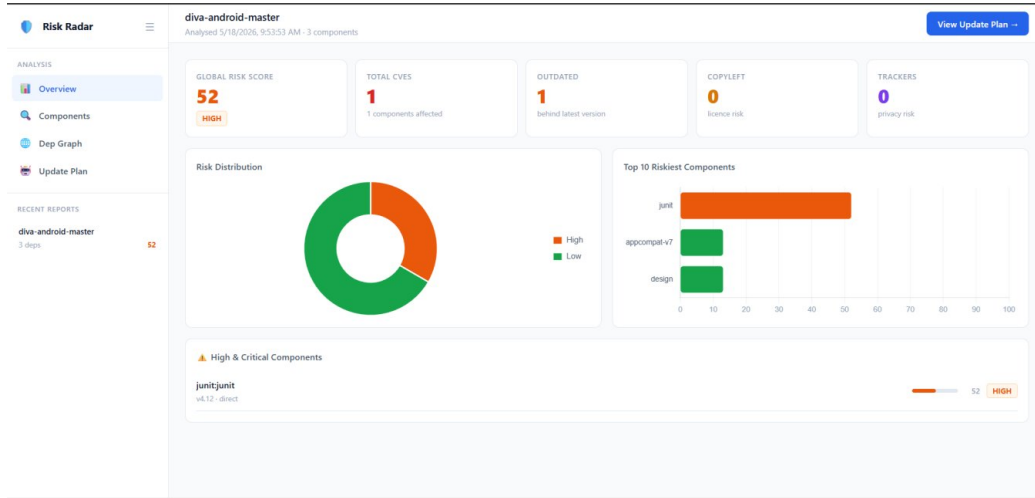


Figure 4: DRR overview dashboard for DIVA Android: global risk score 52/100 (High), 1 CVE across 1 affected component, 1 outdated dependency.

Figure 4 shows the overview dashboard for DIVA. The global risk score of **52/100 (High)** is driven entirely by `junit:junit:4.12`, which carries CVE GHSA-tpjf-t59h-4vtr (CVSS 5.9) and is 14 versions behind the current release. The two remaining dependencies (`appcompat-v7`, `design`) remain in the Low band.

Table 4: DIVA Android — per-component risk summary (DRR).

Component	Score	CVEs	Risk Level
junit:junit:4.12	52	1	High
appcompat-v7	13	0	Low
design	13	0	Low

3.3. OWASP UnCrackable Level 2

UnCrackable L2 is a binary reverse-engineering challenge from the OWASP Mobile Security Testing Guide [5]. Its APK embeds 14 dependencies, including analytics SDKs whose version metadata is not preserved in the compiled binary.

COMPONENT	VERSION	LATEST	SCORE	CVES	LICENCE	LEVEL
jackson-databind com.fasterxml.jackson.core	unknown	2.18.4	64.2	69	UNKNOWN	HIGH
analytics com.urgent.analytics.android	unknown	4.11.3	25	—	UNKNOWN	MODERATE
core com.google.zxing	unknown	3.5.3	23	—	UNKNOWN	MODERATE
rxjava io.reactivex.rxjava3	unknown	3.1.10	19.2	—	UNKNOWN	LOW
mixpanel-android com.mixpanel.android	unknown	?	14.2	—	UNKNOWN	LOW
material com.google.android.material	unknown	?	13	—	UNKNOWN	LOW
fragment androidx.fragment	unknown	?	13	—	UNKNOWN	LOW
design com.android.support	unknown	?	13	—	UNKNOWN	LOW
core-ktx androidx.core	unknown	?	13	—	UNKNOWN	LOW
appcompat androidx.appcompat	unknown	?	13	—	UNKNOWN	LOW

Figure 5: Component table for OWASP UnCrackable L2 (14 components, sorted by risk score). `jackson-databind` (score 64.2, 69 CVEs) is classified High.

Figure 5 displays the full component table. With a global score of **64.2/100 (High)**, the project is primarily threatened by `jackson-databind` (unknown version), which matches 69 CVEs in the combined OSV/NVD database.

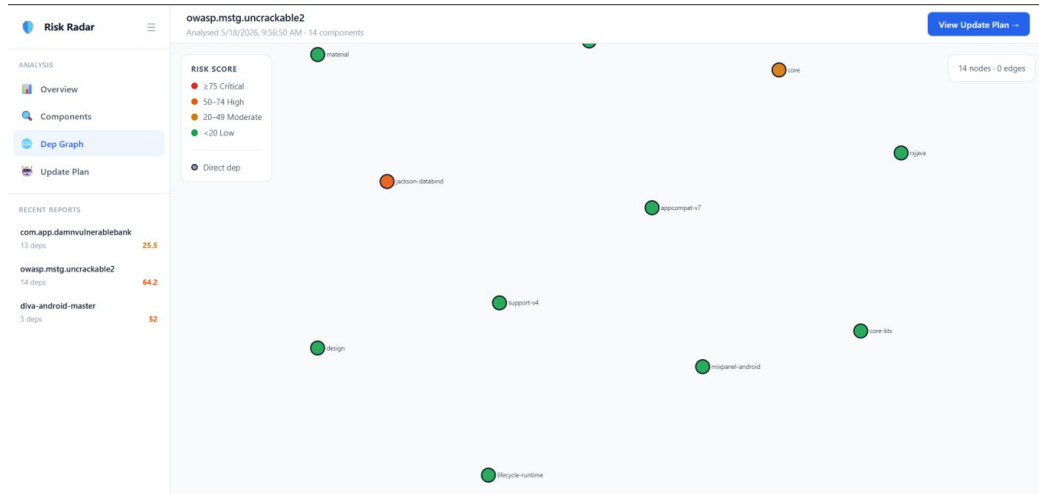


Figure 6: Force-directed dependency graph for OWASP UnCrackable L2 (14 nodes). Node colour encodes risk level: orange = High, yellow = Moderate, green = Low.

3.4. *DamnVulnerableBank*

DamnVulnerableBank is a modern intentionally insecure banking application using a contemporary Gradle setup with Firebase, Volley, and RxJava.

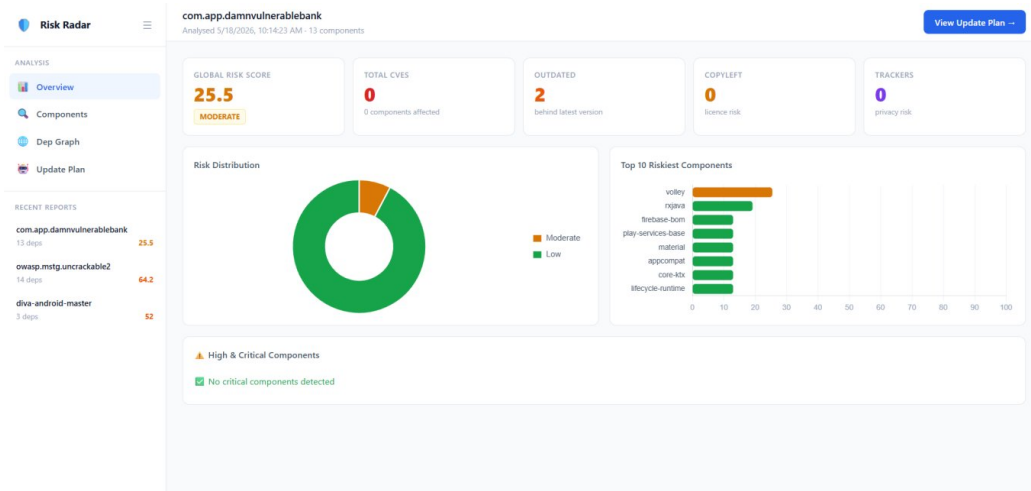


Figure 7: DRR overview for DamnVulnerableBank: global score 25.5/100 (Moderate), 0 CVEs, 2 outdated dependencies, 0 copyleft or tracker detections.

As shown in Figure 7, DamnVulnerableBank achieves the lowest global score (**25.5/100, Moderate**). No CVEs are recorded against its 13 declared dependencies; the score is attributable to two outdated libraries (volley and rxjava).

3.5. AI-Powered Update Plan

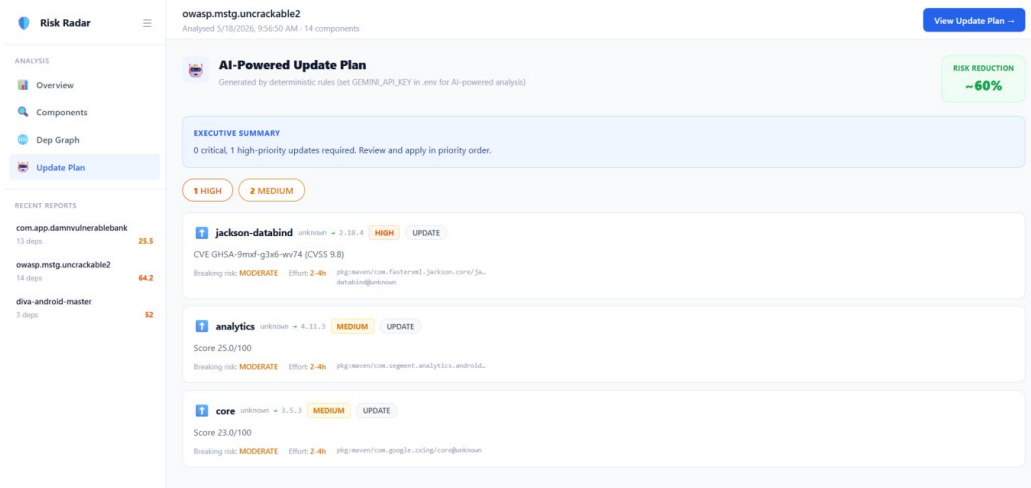


Figure 8: AI-generated Update Plan for OWASP UnCrackable L2: 1 High-priority update (jackson-databind → 2.18.4, CVE GHSA-9mxf-g3x6-wv74, CVSS 9.8) and 2 Medium-priority updates. Projected risk reduction $\approx 60\%$.

Figure 8 presents the AI-generated remediation plan for UnCrackable L2. Following the three recommended updates in priority order reduces the projected global risk score from 64.2 to approximately 25.7 — a reduction of **60%** — without introducing high-breaking-risk changes.

3.6. Benchmark Summary

Table 5: Benchmark comparison across the three analysed projects (DRR results).

Project	Deps	Score	Level	CVEs	Outdated	Risk red.
DIVA Android	3	52.0	High	1	1	~48%
UnCrackable L2	14	64.2	High	69	0	~60%
DamnVulnerableBank	13	25.5	Moderate	0	2	~14%

4. Comparative Study with Existing APK Analysis Tools

4.1. Scope and Methodology

A number of tools analyse Android applications at the APK level or at the Gradle dependency level. We compare DRR against five widely-used representatives across five capability dimensions: dependency-level CVE detection, multi-criteria risk scoring, transitive dependency graph, SBOM generation, and AI-powered remediation. The tools evaluated are MobSF [6], OWASP Dependency-Check [7], Snyk [8], APKLeaks [9], and apkanalyzer [10].

4.2. MobSF Static Analysis Results on Benchmark APKs

To provide a concrete, data-driven comparison, we ran MobSF v4.5.0 on the same three benchmark APKs used to evaluate DRR. Figures 9a, 9b, and 9c present the MobSF summary scores.

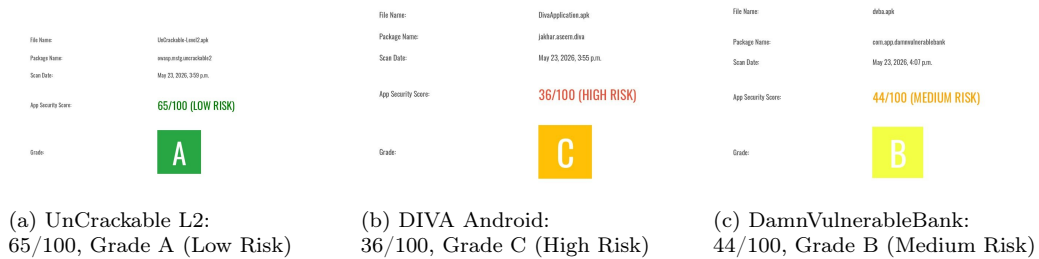


Figure 9: MobSF v4.5.0 security scores for the three benchmark APKs (scanned May 23, 2026).

These results reveal a fundamental difference in scoring philosophy. MobSF assigns UnCrackable L2 a *Low Risk* grade (65/100) despite the presence of 69 CVEs in its dependency tree, because MobSF focuses on code-level and manifest-level findings (anti-debug, Janus vulnerability, binary hardening) rather than library vulnerability matching. DRR, by contrast, rates the same application as *High* (64.2/100) due to `jackson-databind`'s extensive CVE exposure.

For DIVA Android, MobSF correctly identifies significant code-level risks — debug certificate, debuggable manifest flag, SQL injection patterns, NX-disabled native libraries — and assigns a High Risk grade (36/100). DRR assigns a High score (52/100) but for a different reason: a single dependency CVE (`junit:junit:4.12`). The MobSF findings for DIVA include: 5 HIGH severity issues (debug certificate, Janus vulnerability, NX-disabled binaries, no stack canaries), 7 MEDIUM findings, and 3 abused permissions (`WRITE_EXTERNAL_STORAGE`, `READ_EXTERNAL_STORAGE`, `INTERNET`).

For DamnVulnerableBank, MobSF reports 6 HIGH severity findings — cleartext traffic enabled, insecure network security configuration trusting user certificates, two App Link misconfigurations, a missing stack canary, and an unpatched minSdk — and assigns Medium Risk (44/100). DRR reports Moderate risk (25.5/100) due to version staleness. Notably, MobSF uncovered a hardcoded Firebase API key and a live Firebase database URL (`https://damn-vulnerable-bank.firebaseio.com`), findings entirely outside DRR's dependency-focused scope.

Table 6 summarises the complementary findings across both tools for all three benchmarks.

Table 6: Comparison of DRR vs. MobSF findings on the three benchmark APKs.

Finding type	DIVA	UnCrackable L2	DamnVulnerableBank
<i>DRR results</i>			
Global risk score	52 (High)	64.2 (High)	25.5 (Moderate)
Dep. CVEs detected	1	69	0
Outdated deps	1	0	2
Tracker detections	0	0	0
Copyleft licences	0	0	0
<i>MobSF results (v4.5.0)</i>			
App security score	36/100 (C)	65/100 (A)	44/100 (B)
High severity findings	5	1	6
Medium severity findings	7	3	9
Hardcoded secrets	1	0	1 (Firebase key)
Anti-debug/anti-VM code	No	Yes	Yes
NX-disabled binaries	Yes (7/7 libs)	No (all NX enabled)	No
Missing stack canary	Yes (6/7 libs)	No	Yes (3 libs)
Debug certificate	Yes	No	No
Cleartext traffic	No	No	Yes

4.3. Feature Comparison Matrix

Table 7 presents a comprehensive feature matrix comparing DRR against all five evaluated tools.

Table 7: Feature comparison of DRR against leading APK and SCA tools.

Feature	DRR	MobSF	Dep-Check	Snyk	apkanalyzer
APK ingestion	Yes	Yes	—	Partial	Yes
Gradle ingestion	Yes	—	Yes	Yes	—
Dependency-level CVE	Yes	Partial	Yes	Yes	—
Transitive dep. graph	Yes	—	Partial	Yes	—
Multi-criteria score	Yes	—	—	—	—
Tracker detection	Yes	Partial	—	—	—
Licence compliance	Yes	—	—	Yes	—
SBOM export (CycloneDX)	Yes	—	Yes	Yes	—
SBOM export (SPDX)	Yes	—	—	Yes	—
AI-powered update plan	Yes	—	—	Partial	—
Self-hosted / open-source	Yes	Yes	Yes	—	Yes
Interactive web dashboard	Yes	Yes	—	Yes	—
Code-level static analysis	—	Yes	—	Partial	—
Binary hardening analysis	—	Yes	—	—	—
Hardcoded secret detection	—	Yes	—	Partial	—

4.4. In-Depth Comparison with MobSF

MobSF is the closest open-source alternative to DRR in scope. Both tools accept APK files, provide a web-based dashboard, and are self-hostable. However, their security focus differs fundamentally.

MobSF is a *code security scanner*: it performs static analysis of Dalvik bytecode, inspects `AndroidManifest.xml` for dangerous permissions, flags insecure API usages (hardcoded keys, weak ciphers, SQL injection), analyses native binary hardening (NX bit, stack canary, RELRO, FORTIFY), and can optionally run dynamic analysis in an emulator. As shown in the benchmark results, MobSF correctly identified the debug certificate and NX-disabled binaries in DIVA, the anti-debug/anti-VM code in UnCrackable L2, and the Firebase API key leakage and cleartext traffic configuration in DamnVulnerableBank. These are findings entirely outside DRR’s scope.

DRR is a *supply chain security scanner*: it focuses on the dependency layer, systematically querying OSV, NVD, Maven Central, and Exodus Privacy for each detected artefact, computing a four-dimensional risk

score, and producing standards-compliant SBOMs. Its identification of 69 CVEs against `jackson-databind` in UnCrackable L2 — missed entirely by MobSF — demonstrates its complementary value. The two tools are therefore *complementary* rather than competing. A robust Android security pipeline should run both: MobSF for code and configuration risks, DRR for third-party dependency supply chain risks.

4.5. Comparison with OWASP Dependency-Check

OWASP Dependency-Check (ODC) uses NVD CPE matching to identify known vulnerabilities in Java JARs. Its main limitations in an Android context are: (a) no native Gradle task for Android modules and no APK support; (b) CVE detection relies on CPE string matching, which produces a higher false-positive rate than the PURL-based OSV queries used by DRR; (c) no tracker detection, no licence scoring, and no AI-assisted remediation.

4.6. Comparison with Snyk

Snyk offers best-in-class dependency scanning for Gradle and Maven projects and supports CycloneDX/SPDX SBOM export. Its limitations are: (a) it is a commercial SaaS product with no self-hosted option in its free tier; (b) it does not support direct APK ingestion; (c) no integrated tracker detection; (d) its AI-assisted features focus on code fixes rather than dependency update planning.

4.7. Positioning Summary

DRR occupies a unique position in the Android security tooling landscape: it is the only open-source, self-hosted tool that simultaneously covers APK and Gradle ingestion, four-dimensional risk scoring, CycloneDX + SPDX SBOM generation, transitive dependency graph analysis, Exodus-based tracker detection, and an AI-powered update plan — accessible through a web dashboard and a local CLI.

5. Impact

DRR addresses a measurable gap in the Android security toolchain. Its direct beneficiaries are three practitioner groups.

Android developers gain a zero-configuration tool that answers the three core supply chain questions — which dependencies are vulnerable, how severe is the aggregate risk, and what should be updated first — within a single analysis run.

Security engineers and auditors benefit from the standards-compliant SBOM output (CycloneDX 1.5, SPDX 2.3) that feeds directly into procurement workflows, supplier questionnaires, and regulatory compliance reports (e.g. EU Cyber Resilience Act, NIST SSDF). The four-dimensional risk model provides a richer signal than raw CVE counts alone, incorporating licence exposure and tracker privacy risk that purely vulnerability-focused tools miss.

DevSecOps teams can integrate DRR into local security workflows via the CLI to enforce risk thresholds before deployments, shifting security feedback left without requiring manual security reviews for each release.

The three-benchmark evaluation demonstrates concrete risk reduction: following DRR’s recommended update sequence reduces the global risk score by up to 60% in a single sprint, with each individual update estimated at 2–4 hours of developer effort.

6. Conclusions

This paper presented **Dependency Risk Radar**, an open-source, self-hosted supply chain security platform for Android and Java applications. DRR combines four complementary capabilities: automated dependency extraction from Gradle files and APK binaries, multi-criteria risk scoring across CVE severity, version obsolescence, licence compliance, and embedded tracker detection, transitive risk propagation through a directed acyclic dependency graph, and an AI-assisted prioritised update plan.

Evaluation on three Android security benchmarks demonstrates practical utility. On OWASP UnCrackable L2, following the AI-generated update plan reduces the projected global risk score by 60%. On DIVA Android, the model correctly isolates a single High-risk dependency amid otherwise Low-risk libraries. On DamnVulnerableBank, the tool surfaces staleness-driven risk invisible to pure CVE-matching tools.

The comparative analysis confirms that DRR and MobSF serve complementary roles: MobSF excels at code-level static analysis, binary hardening assessment, and secret detection, while DRR provides the supply chain layer — systematic CVE matching, four-dimensional scoring, and SBOM generation — that MobSF does not address. Together they form a comprehensive Android security assessment pipeline.

Future work will focus on: (i) version fingerprinting for APK binaries to resolve the **unknown** version limitation; (ii) reachability analysis to eliminate false positives for CVEs in unused code paths; (iii) Kotlin

Multiplatform and Flutter/Dart support; and (iv) a longitudinal study correlating DRR risk scores with real-world exploitation events to empirically calibrate the scoring weights.

References

- [1] M. Linares-Vasquez *et al.*, “How Do API Changes Trigger Stack Overflow Discussions? A Study on the Android SDK,” *Proc. IEEE SANER*, 2016.
- [2] P. Bhatt, “DIVA Android — Damn Insecure and Vulnerable App,” <https://github.com/payatu/diva-android>, 2022.
- [3] OWASP Foundation, “Mobile Security Testing Guide — UnCrackable Apps,” <https://github.com/OWASP/owasp-mastg>, 2024.
- [4] R. Tammana, “Damn Vulnerable Bank,” <https://github.com/rewanthtammana/Damn-Vulnerable-Bank>, 2022.
- [5] OWASP Foundation, “Mobile Application Security Testing Guide (MASTG),” <https://mas.owasp.org/>, 2024.
- [6] A. Abraham, “Mobile Security Framework (MobSF),” <https://mobsf.github.io/Mobile-Security-Framework-MobSF/>, 2024.
- [7] J. Long *et al.*, “OWASP Dependency-Check,” <https://owasp.org/www-project-dependency-check/>, 2024.
- [8] Snyk Ltd., “Snyk Open Source Security,” <https://snyk.io/product/open-source-security-management/>, 2024.
- [9] W. Daldoul, “APKLeaks,” <https://github.com/dwisiswant0/apkleaks>, 2024.
- [10] Google Inc., “apkanalyzer — Android SDK command-line tool,” <https://developer.android.com/tools/apkanalyzer>, 2024.
- [11] O. Chang *et al.*, “OSV: A Scalable, Open Vulnerability Database,” <https://osv.dev/>, 2021.
- [12] OWASP Foundation, “CycloneDX Software Bill of Materials Standard,” <https://cyclonedx.org/>, 2024.
- [13] Linux Foundation, “SPDX — Software Package Data Exchange,” <https://spdx.dev/>, 2024.

- [14] Exodus Privacy, “Exodus Privacy Tracker Database,” <https://exodus-privacy.eu.org/>, 2024.
- [15] NIST, “National Vulnerability Database,” <https://nvd.nist.gov/>, 2024.